



CE 222 – Computer Organization and Assembly Language (COAL)

Lecture 27

Dr. Asad Mahmood

April 07, 2026

Lecture Outline

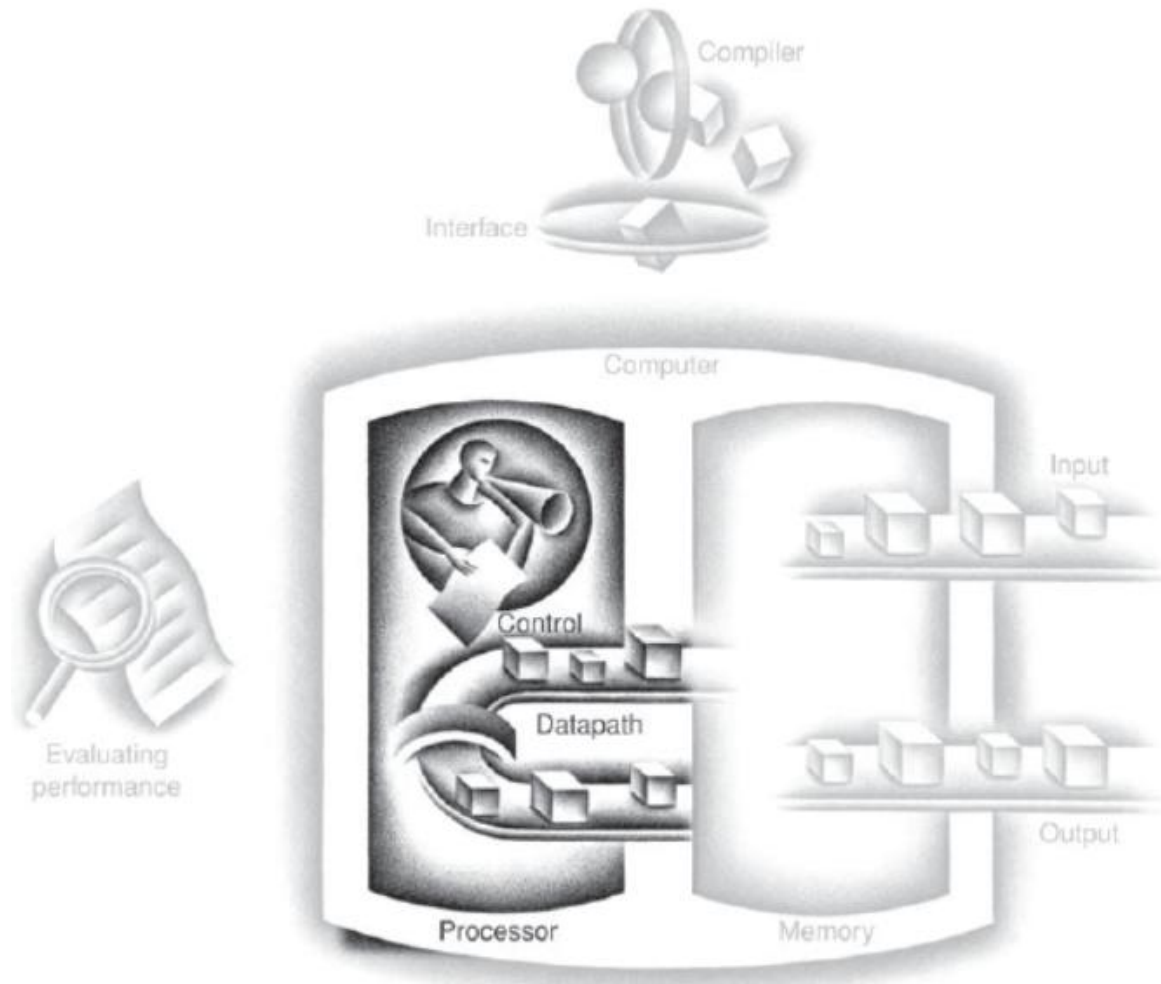
- Announcements (Assignment and Quiz 3)
- Outline of Previous Lecture:
 - **Chapter 4 – The Processor**
 - 4.3 Building a Datapath
- Outline of Today's Lecture:
 - **Chapter 4 – The Processor**
 - 4.4 A Simple Implementation Scheme

Chapter 4

The Processor

Chapter Theme

The Five Classic Components of a Computer



Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

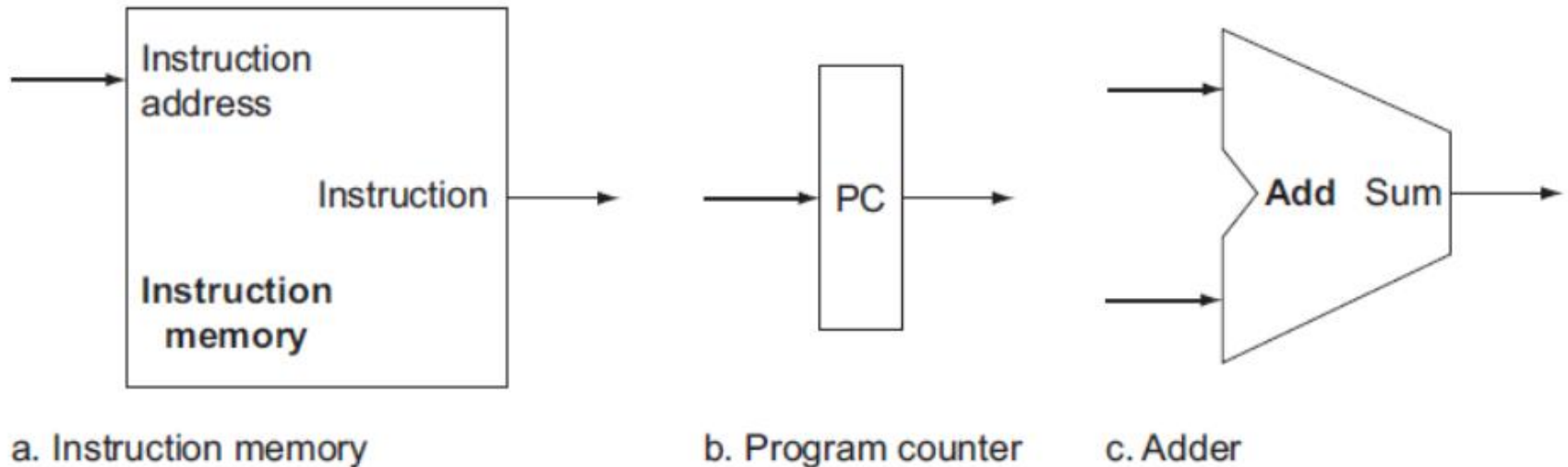
Building a Datapath

- Datapath
 - Set of elements that process data and addresses in the CPU
 - Registers, ALUs, mux's, memories, ...
- We will build a RISC-V datapath incrementally
 - Refining the overview design

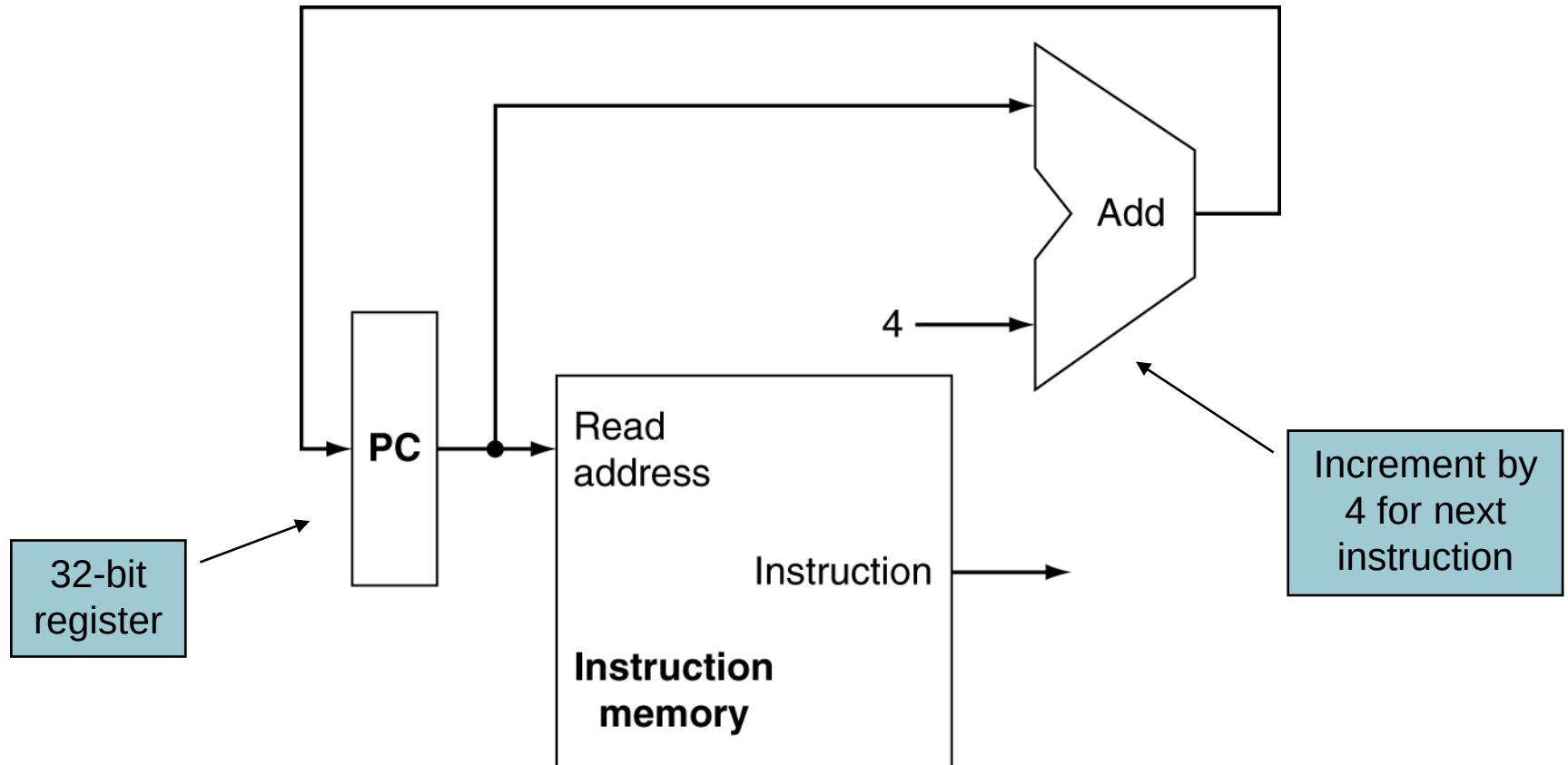
Datapath for Instruction Fetch

Datapath Elements

- PC width

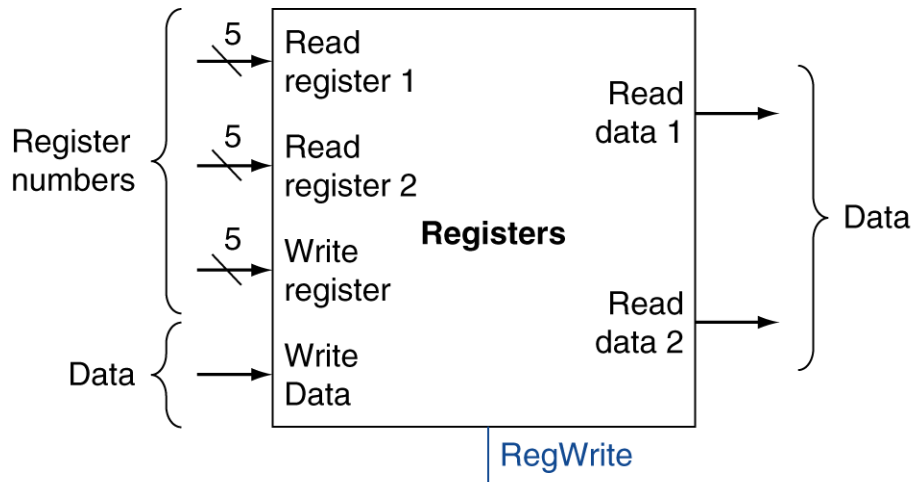


Datapath for Instruction Fetch

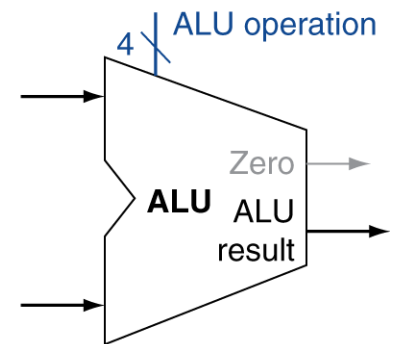


R-type Instructions

- Steps in executing R-type instructions:
 1. Read two register operands
 2. Perform arithmetic/logical operation
 3. Write result in register-file
- I/Os for the register-file



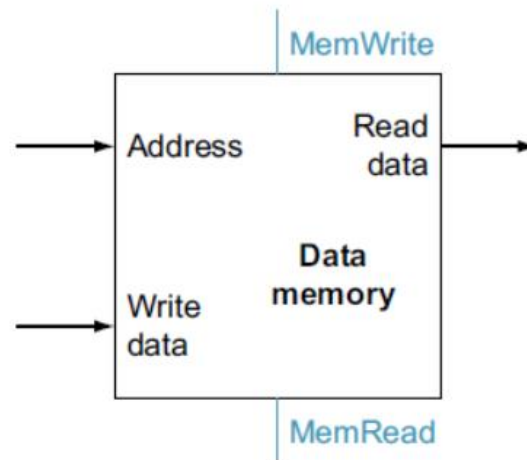
a. Registers



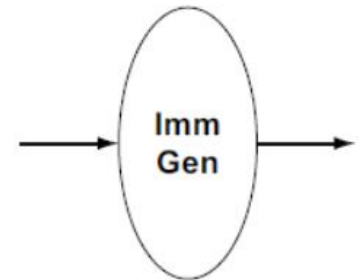
b. ALU

Memory access Instructions

- Steps in executing memory-access (load, store) instructions:
 1. Read register operands
 2. Calculate address using 12-bit offset
 - Use ALU, but sign-extend offset
 3. Load: Read memory and update register
 4. Store: Write register value to memory
- I/Os for the datapath elements

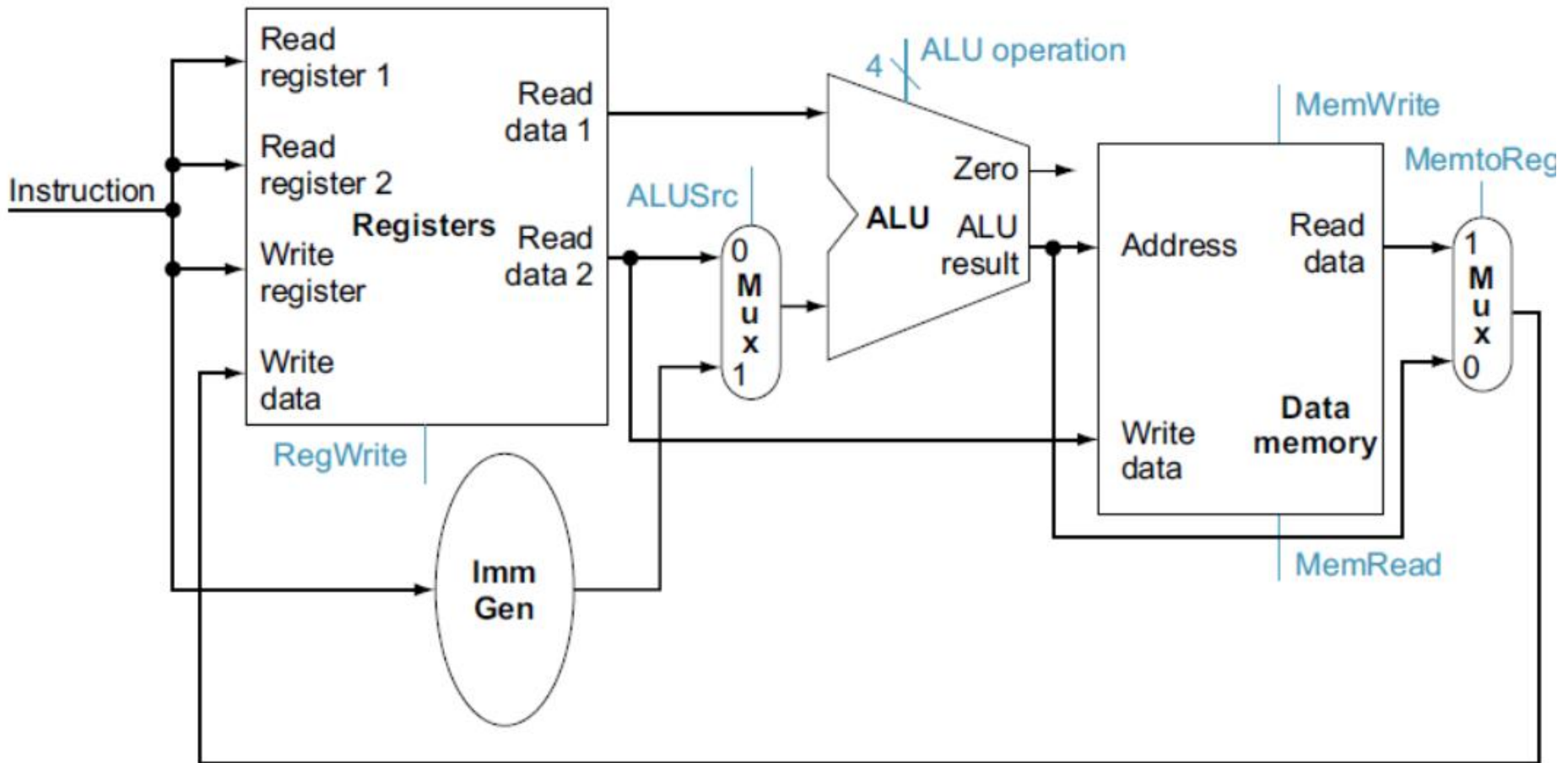


a. Data memory unit



b. Immediate generation unit

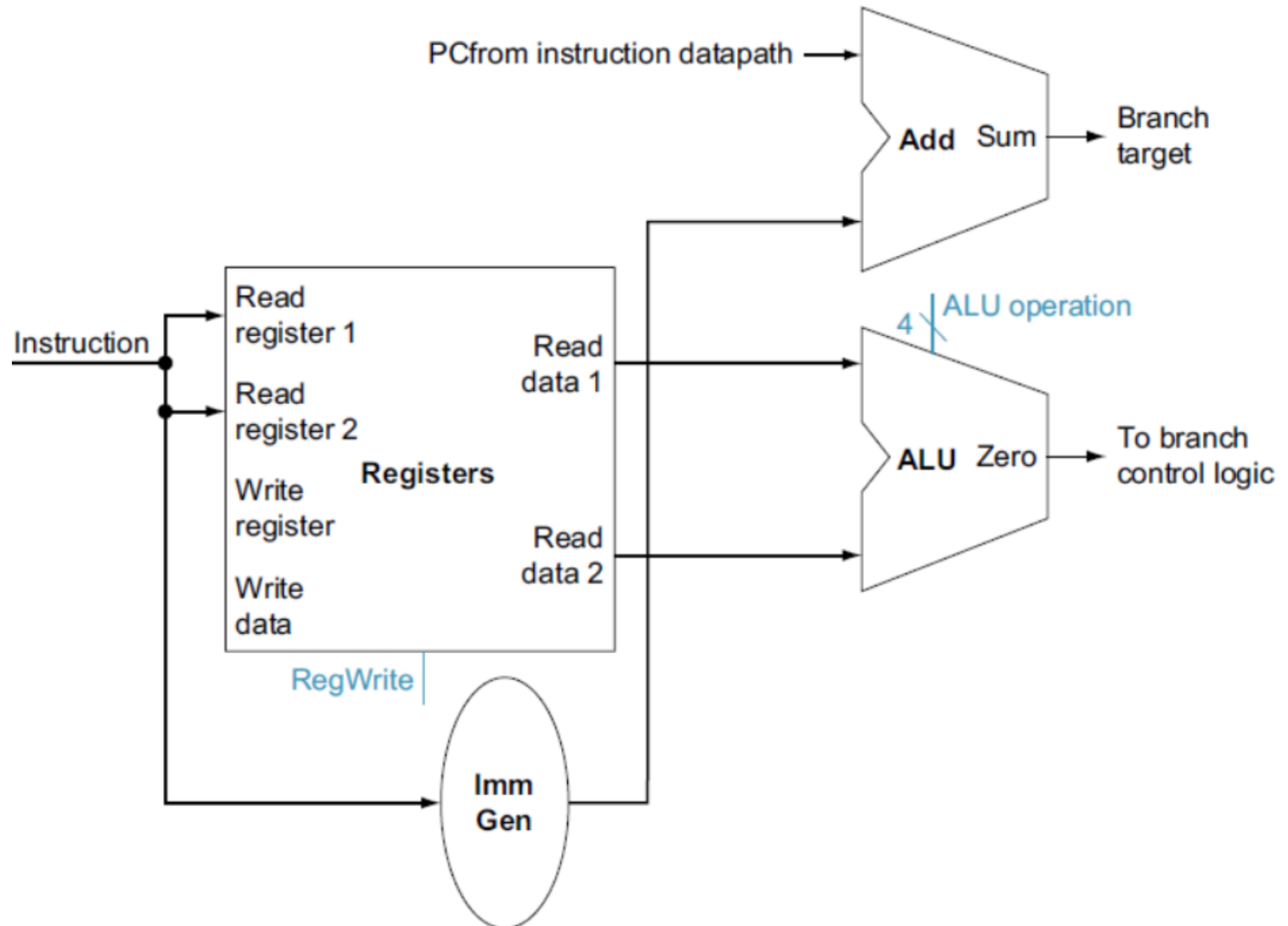
R-type + Memory-access Instructions datapath



Branch Instructions

- Steps in executing branch instructions:
 1. Read register operands
 2. Compare operands
 1. Use ALU, subtract and check Zero output
 3. Calculate target address
 - Sign-extend offset
 - Shift left 1 place (halfword displacement)
 - Add to PC value

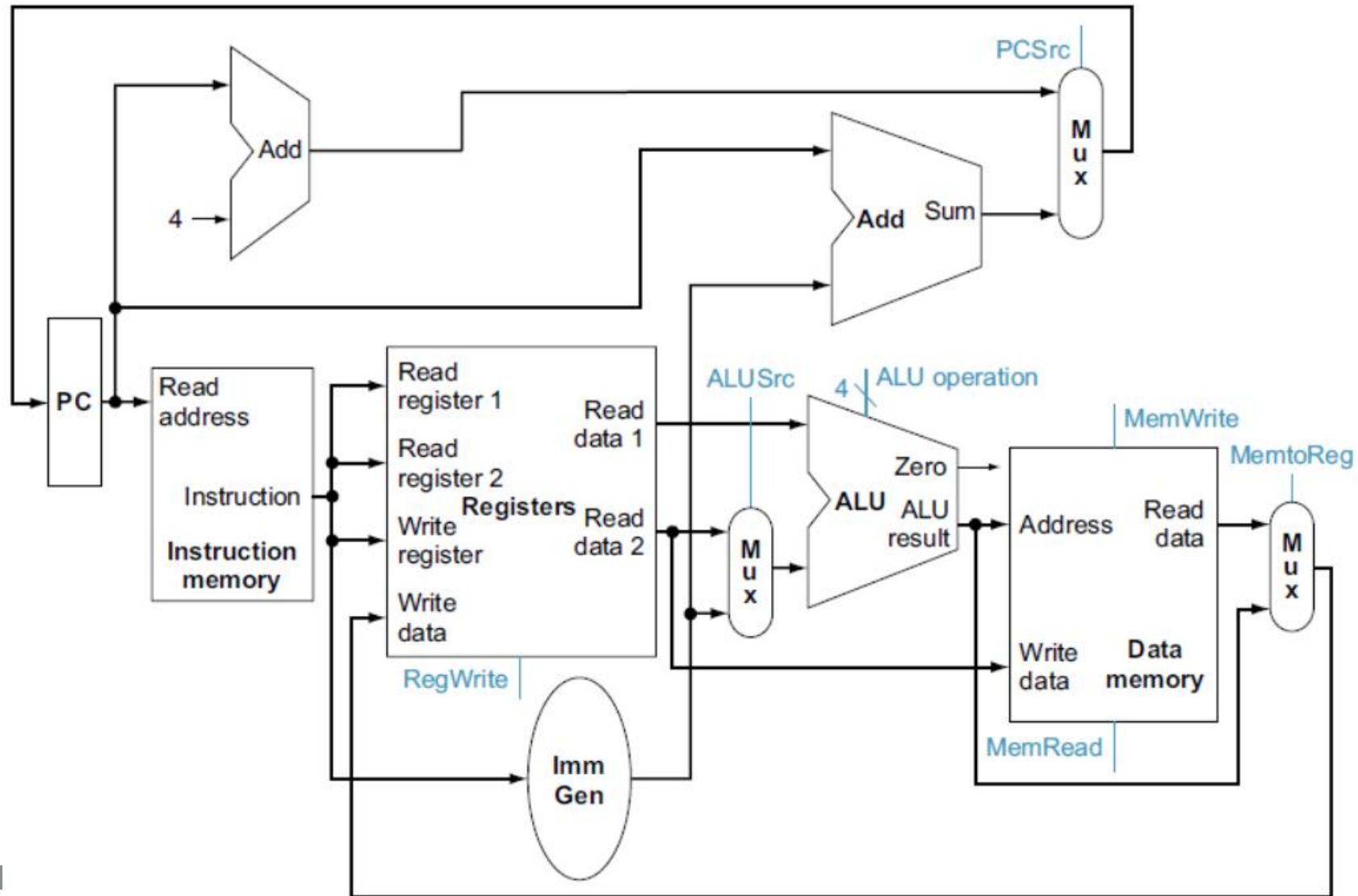
Branch Instructions datapath



Integrated datapath

- Integrating datapaths for instruction fetch, arithmetic/R-type, memory-access, branch instructions into a single datapath
- Use multiplexers where alternate data sources are used for different instructions

Integrated Datapath



Integrated datapath

- Single-cycle processor
 - All steps for an instruction must happen in the right sequence -> each datapath element can only do one function at a time
 - We need separate instruction and data memories
 - This data path executes an instruction in one clock cycle

Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

ALU Control

- Let us make a simple implementation of our **RISC-V subset** by **adding control** to the datapath we made in the last section.
- We will start by developing **control for ALU**
- ALU used in **arithmetic and logical instructions**
- Role of ALU in **memory-access and branch instruction**.

ALU control	Function
0000	AND
0001	OR
0010	add
0110	subtract

ALU Control

- What can be used to define 'ALU control' bits?
- Assume 2-bit **ALUOp** derived from opcode
 - Combinational logic derives ALU control

Instruction opcode	ALUOp	Operation	Funct7 field	Funct3 field	Desired ALU action	ALU control input
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001
lw	00	load word	XXXXXXXX	XXX	add	0010
sw	00	store word	XXXXXXXX	XXX	add	0010
beq	01	branch if equal	XXXXXXXX	XXX	subtract	0110

ALU Control

- Recalling instruction formats, as studied in chap. 2

Name (Bit position)	Fields					
	31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

ALU Control

- Design of control unit can be done using a truth table
- Don't care terms

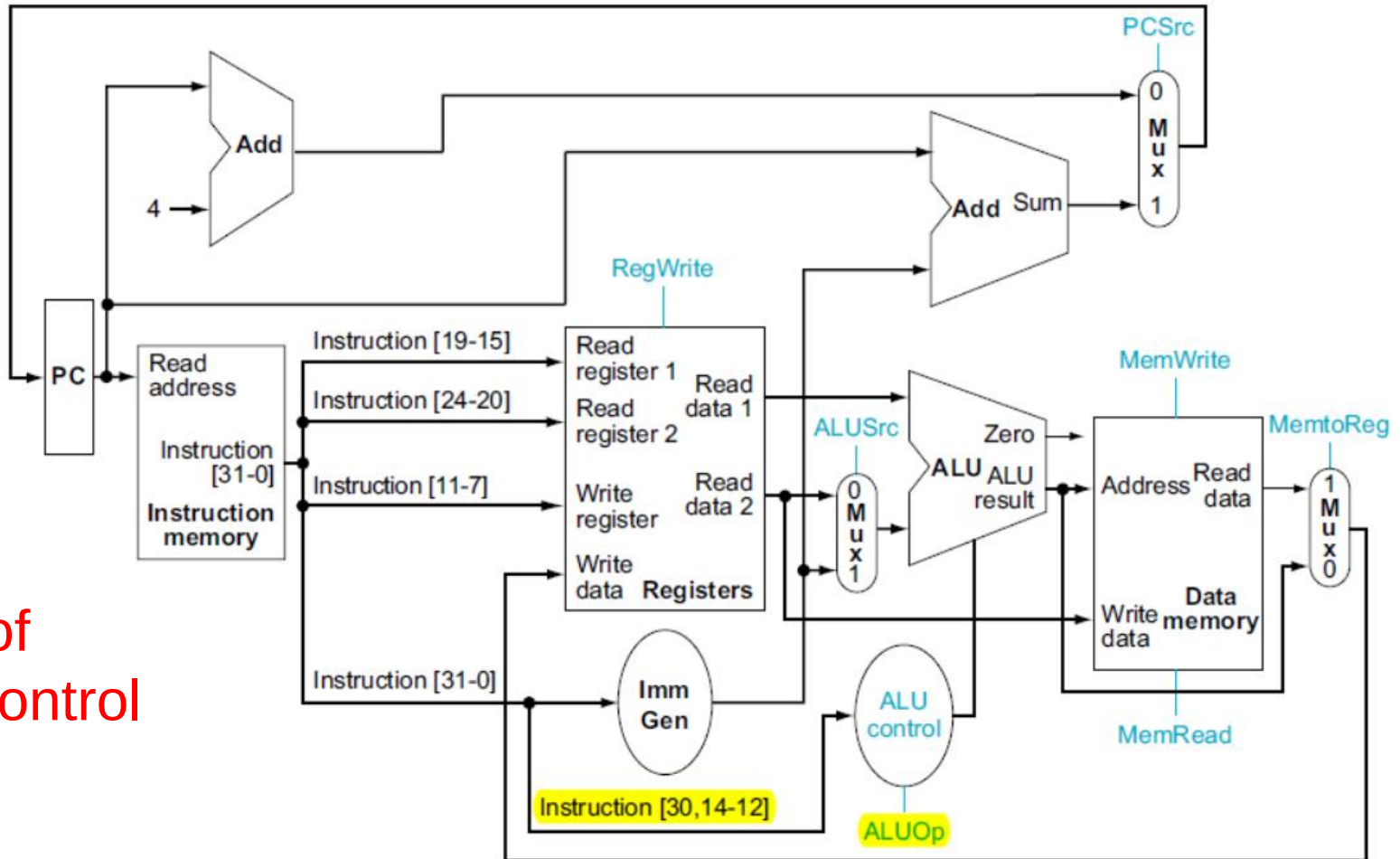
ALUOp		Funct7 field							Funct3 field			Operation
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

The Main Control Unit

- Let us now look at the design of the complete control unit of our simplified processor
- Important to see our target instruction formats and from where to get inputs for the control unit e.g. opcode
- Via Regularity, RISC-V instruction formats help simplify hardware design, as compared to other architectures e.g. MIPS

The Main Control Unit

- Datapath with all control signals



Function of
different control
signals

The Main Control Unit

- Function of different control signals in our simplified processor

Signal name	Effect when deasserted	Effect when asserted
RegWrite	None.	The register on the Write register input is written with the value on the Write data input.
ALUSrc	The second ALU operand comes from the second register file output (Read data 2).	The second ALU operand is the sign-extended, 12 bits of the instruction.
PCSrc	The PC is replaced by the output of the adder that computes the value of $PC + 4$.	The PC is replaced by the output of the adder that computes the branch target.
MemRead	None.	Data memory contents designated by the address input are put on the Read data output.
MemWrite	None.	Data memory contents designated by the address input are replaced by the value on the Write data input.
MemtoReg	The value fed to the register Write data input comes from the ALU.	The value fed to the register Write data input comes from the data memory.

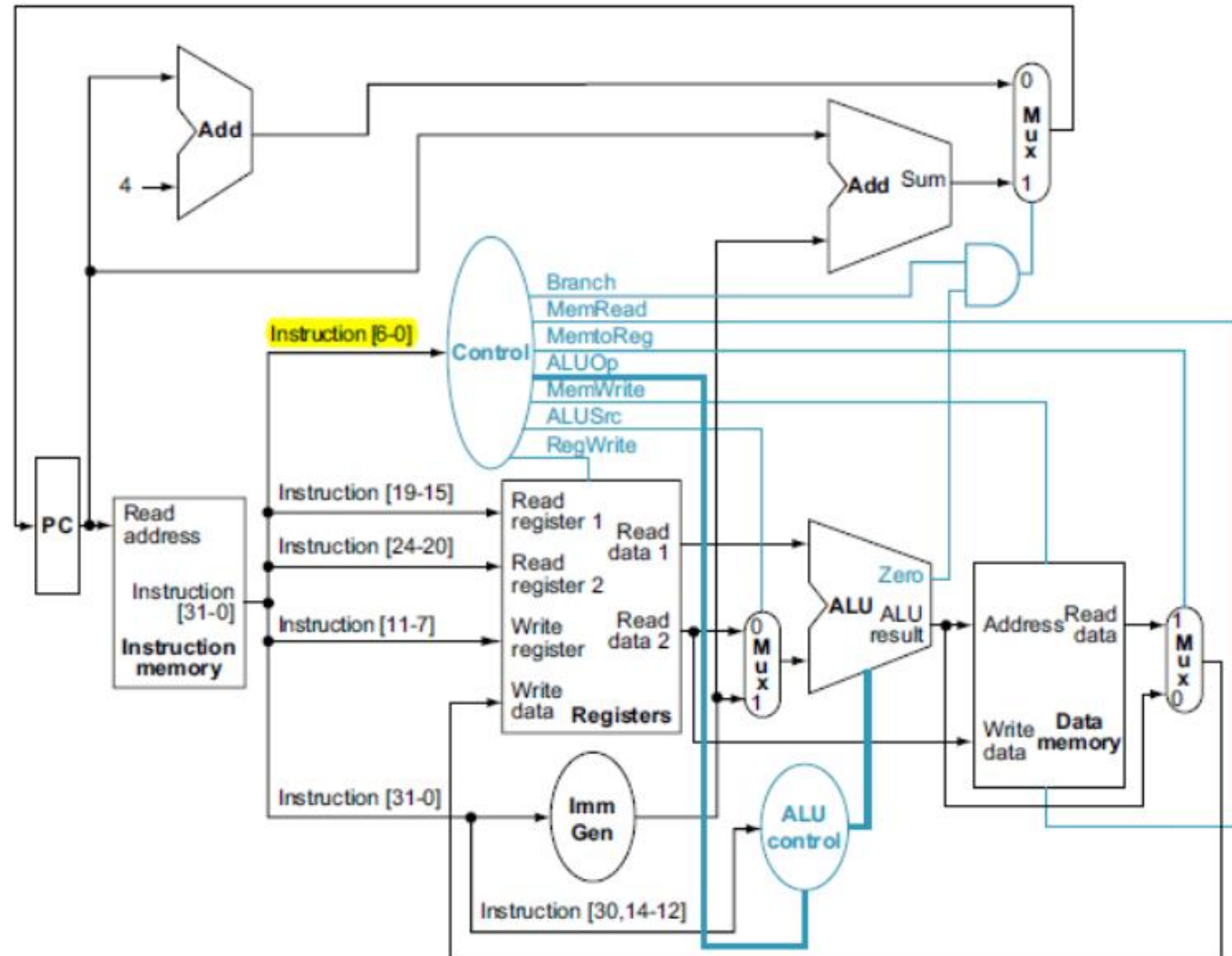
The Main Control Unit

- Values of control signals for different instruction types

Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

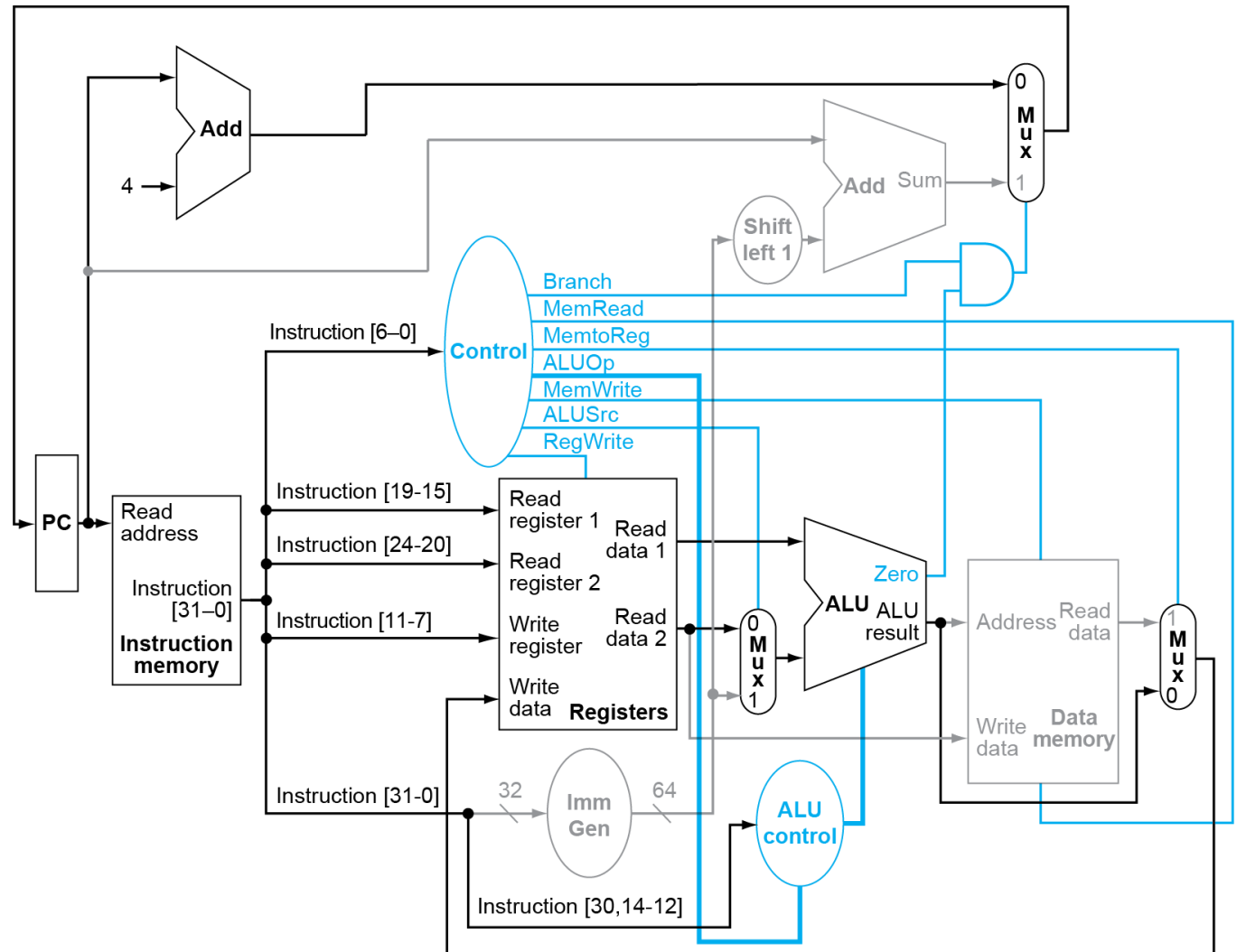
Datapath With Control

The Control Unit sets all the control signals except one!



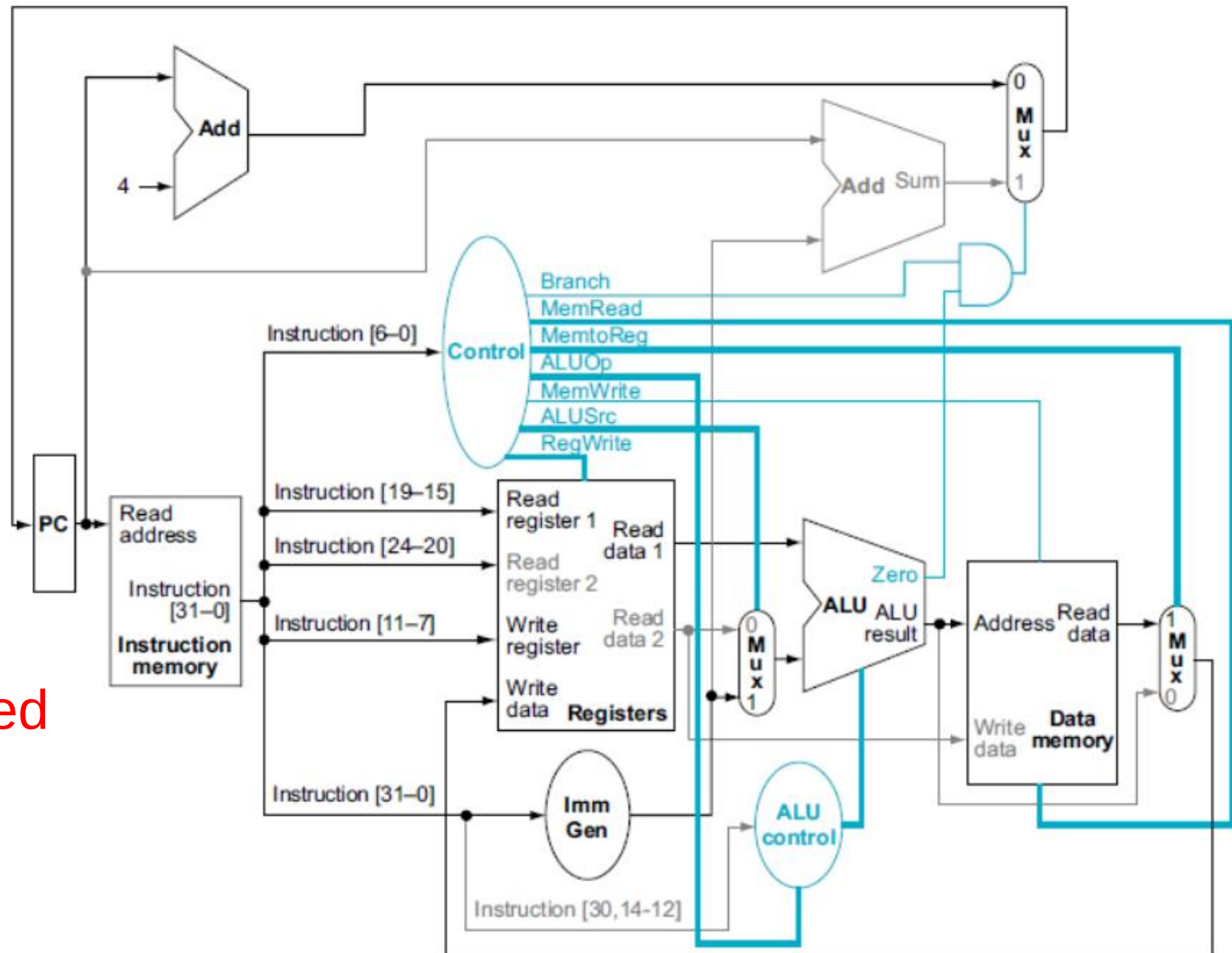
R-Type Instruction

- Instruction Execution steps
- Clock cycles

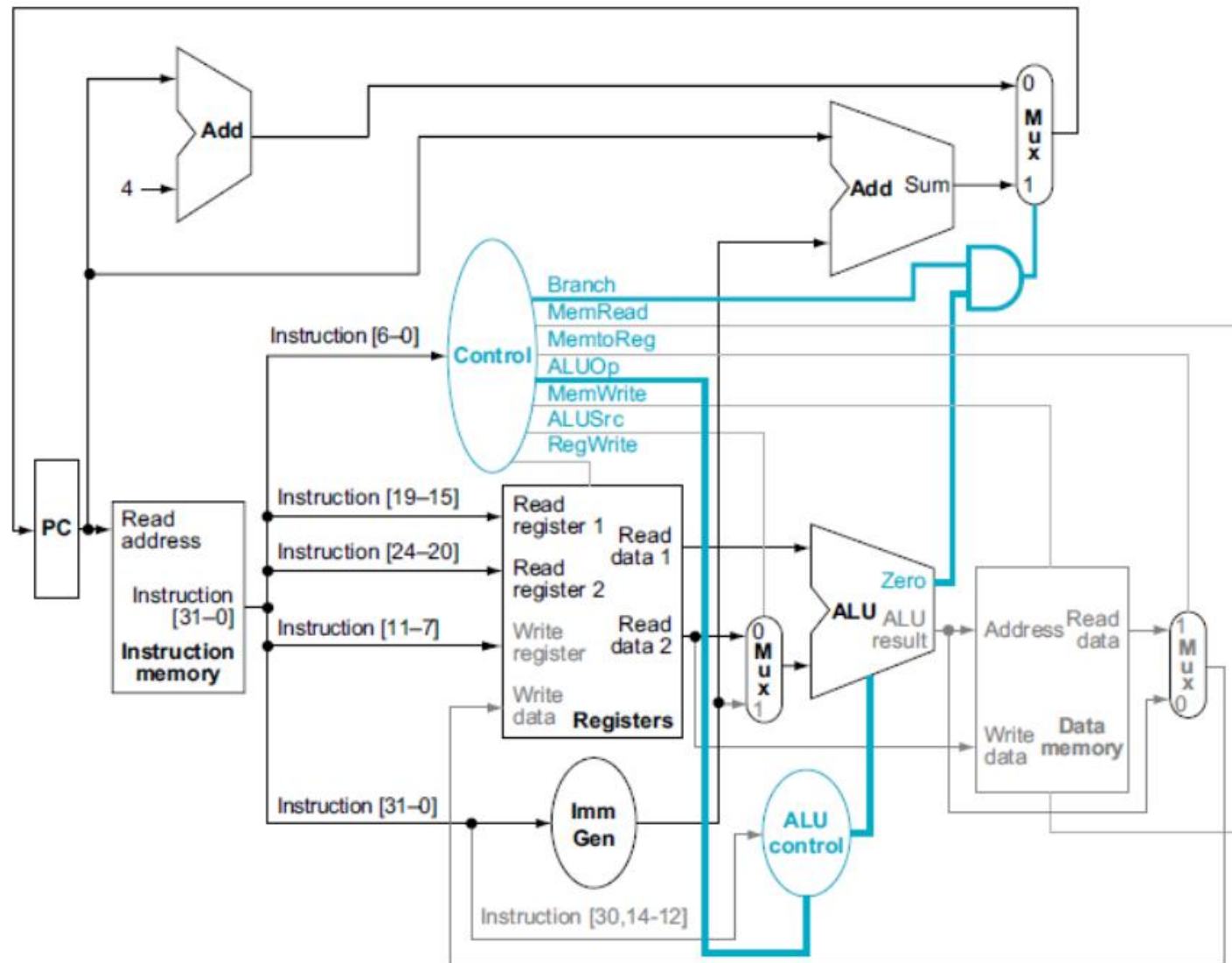


Load Instruction

- Highlighted control lines



Branch-on-Equal Instruction



Performance Issues

- Longest delay determines clock period
 - Critical path: load instruction
 - CPI ?
- Not feasible to vary period for different instructions
 - Violates design principle of *Making the common case fast*
- We will improve performance by pipelining

Chapter Outline

Chapter 4 – The Processor

- 4.1 Introduction
- 4.2 Logic Design Conventions
- 4.3 Building a Datapath
- 4.4 A Simple Implementation Scheme
- 4.5 Multicycle Implementation
- 4.6 An Overview of Pipelining
- 4.7 Pipelined Datapath and Control
- 4.8 Data Hazards: Forwarding versus Stalling
- 4.9 Control Hazards
- 4.10 onwards (self-reading)

Introduction

- Single cycle implementation
- In a multicycle implementation, each *step* in the execution will take 1 clock cycle.
- Multicycle implementation has the following two advantages:
 - Different instructions to execute in different no. of clock cycles
 - Ability to share/re-use functional units within the execution of a single instruction

Multicycle datapath

- High-level view of the multicycle datapath
- Identify the differences with respect to the single-cycle datapath
- Steps executed in a single clock-cycle in this datapath
- Address of the single memory

